

nff — power characteristics on ESP32

WiFi/MQTT vs. BLE, per operation — and what it costs a battery

ESP32-WROOM-32E • Rev. A • 2026-07-14 • PRELIMINARY

Data status — partially MEASURED (Rev. A.1). Figures marked * are measured on the shunt rig (§6, 2026-07-14) and de-embedded to the module's 3.3 V rail. **Everything else remains modelled**, including *every* BLE figure and *every* sleep floor. Unmarked figures are **typical values, derived by engineering model** at 25 °C and 3.3 V; they are not yet production-tested. The WiFi column is modelled from the ESP32 datasheet's characteristics; the BLE column is derived from it by the published method in §2, so every figure is traceable and falsifiable. **Suitable for design sizing and for ranking operations against one another. Not suitable as acceptance criteria or contractual limits.** §7 specifies the bench procedure that promotes this sheet to characterised data. *Rev. B will carry measured values.*

1 Headline

Transport	Per 5 min cycle	Life on 6 000 mA h	Why
WiFi, always associated	— (* 49.7 mA continuous)	4.3 days	The SDK never calls <code>esp_wifi_set_ps()</code> . The radio simply never goes down.
WiFi, cold-cycled	154 µA h	3.8 months	Associate + DHCP + SNTP + mTLS every wake.
BLE, advertise-only	31.6 µA h	18.4 months	No association. No DHCP. No NTP. No TLS.

BLE does not win on milliamps. It wins by deleting the protocol. Of the 554 mA s a cold-cycled WiFi node spends per wake *, **548 mA s — 99 % — is association, DHCP, NTP and the TLS handshake**, spent *before a single byte of payload moves*. The heartbeat itself is 5.4 mA s: 1.0 %. BLE advertising needs *none* of that setup, because an advertisement is connectionless. **The saving is 4.9× per cycle, and essentially all of it is link bring-up that BLE never performs** — not a cheaper radio. **And the BLE radio is not meaningfully cheaper at all:** for link bring-up the module currents are within 4 % of each other (§2, measured). The whole win is the deleted protocol.

2 Derivation — how the BLE column was obtained

The BLE column is not independently modelled. It is computed from the WiFi column by two ratios, so that every BLE figure is traceable and falsifiable.

(a) Current — a two-term model

The module's active current does *not* scale with the radio, because roughly half of it is not the radio:

$$I_{\text{total}} = \underbrace{I_{\text{core}}}_{\text{CPU @ 80 MHz + PLL + flash ctrl}} + I_{\text{radio}}, \quad I_{\text{core}} = 35.0 \text{ mA} \quad (* \text{ measured; identical for both radios}).$$

	I_{core}	I_{radio}	I_{total}
WiFi, connected idle *	35.0 mA	14.7 mA	49.7 mA
WiFi, link bring-up *	35.0 mA	29.7 mA	64.7 mA
WiFi, bulk RX / OTA *	35.0 mA	119.6 mA	154.6 mA
BLE, BT init modelled	35.0 mA	27 mA	62 mA
BLE, advertising modelled	35.0 mA	40 mA	75 mA
BLE, OTA modelled	35.0 mA	73 mA	108 mA

k_I is not one number. Link bring-up: $62/64.7 = 0.96$. Bulk transfer: $108/150.5 = 0.72$.

Measurement killed the single k_I , and it strengthened the thesis. I_{core} is * 35.0 mA — exactly as modelled, and identical for both radios. But the WiFi *radio* term is not a constant: it spans 14.7 mA (connected, idle) to 119.6 mA (bulk RX). **There is no one k_I ; it must be taken per operation class.**

And for link bring-up there is essentially *no* current advantage at all (62 mA BLE vs * 64.7 mA WiFi — within 4 %). The document originally credited BLE with a 1.5× module-current win. **That win does not exist.** BLE's 12× saving on link bring-up is **entirely time** (0.26 s vs * 3.1 s) — which is precisely what this sheet has claimed all along: **BLE does not win on milliamps. It wins by deleting the protocol.** The measurement did not weaken that argument; it removed the one part of it that was wrong.

(b) Time — a throughput ratio, for bulk transfers only

	Effective throughput (payload to flash)	Note
WiFi, HTTP + flash write *	122 kB/s	Measured: 1 MiB in 8.6 s. Flash-bound — the 1M PHY — this part has no LE 2M PHY .
BLE 4.2, GATT + flash write	15 kB/s	
Ratio	$k_t = 8.1$ — BLE bulk transfer takes eight times as long . It was 1.93 only because the	

(c) The rule — and the trap

Class of operation	Duration	Current	Energy ratio
CPU-bound (ECDSA verify, SHA-256)	unchanged	unchanged	1.00 — crypto does not care about the radio
Protocol/latency-bound (like-for-like handshake)	unchanged	$\times k_I$	0.96 — no meaningful saving *
Throughput-bound (OTA, crash dump, cert rollover)	$\times k_t$	$\times k_I$	5.8 — BLE costs far more *
Operations BLE deletes (assoc, DHCP, SNTP, TLS)	—	—	0 — this, and only this, is the win

The trap: you cannot hold duration constant when you change the PHY. A 1.2 MB OTA over BLE draws 30 % *less* current — for $1.9\times$ as long. Energy is the product, so **a BLE OTA costs 29 % more energy than a WiFi OTA** ($0.68 \times 1.93 = 1.31$). Scaling the current while leaving the time alone yields the opposite conclusion, and it is the single easiest way to mislead a customer about the heaviest operation in the system. (*It remains negligible in absolute terms — §3, note 3.*)

3 Energy per nff operation

Typical values, 25 °C. Energy at the module’s 3.3 V rail, $E = 3.3 \text{ V} \cdot I \cdot t$. BLE column derived from the WiFi column per §2.

Operation	WiFi / MQTT			BLE			ΔE
	t	I	E	t	I	E	
<code>nff_init()</code> NVS, panic hook	12 ms	40 mA	1.6 mJ	12 ms	40 mA	1.6 mJ	1.00×
Link bring-up *assoc+DHCP / BT init	3.10 s	64.7 mA	662 mJ	0.26 s	62 mA	53 mJ	0.06×
Time sync SNTP / gateway stamps it	1.80 s	95 mA	564 mJ	—	—	0 mJ	gone
Secure session mTLS+MQTT / GATT	1.60 s	110 mA	581 mJ	0.15 s	75 mA	37 mJ	0.06×
Heartbeat, <i>one packet</i> 250 B / 27 B	45 ms	120 mA	17.8 mJ	2.5 ms	120 mA	0.99 mJ	0.06×
Heartbeat, <i>delivered</i> 15 adv events	<i>(requires the session above)</i>			1.50 s	42 mA	208 mJ	note 1
Keepalive MQTT PINGREQ / none	12 ms	115 mA	4.6 mJ	—	—	0 mJ	gone
Command auth ECDSA-P256 verify	55 ms	62 mA	11.3 mJ	55 ms	62 mA	11.3 mJ	1.00×
↪ ping / diag reply	30 ms	118 mA	11.7 mJ	2.5 ms	120 mA	1.0 mJ	0.09×
↪ reboot	0.90 s	88 mA	261 mJ	0.90 s	78 mA	232 mJ	0.89×
Crash report 2.5 kB	180 ms	105 mA	62 mJ	320 ms	90 mA	95 mJ	1.53×
Enrollment announce + cert rollover	3.20 s	112 mA	1.18 J	1.50 s	85 mA	421 mJ	0.36×
Failed-link retry one attempt	1.90 s	108 mA	677 mJ	structurally eliminated			note 2
OTA *1.2 MB image	9.8 s	150 mA	4.89 J	80 s	108 mA	28.5 J	5.8×
in charge			0.41 mA h			2.40 mA h	note 3
OTA probation soak 5 s	5.00 s	82 mA	1.35 J	5.00 s	55 mA	0.91 J	0.67×
OTA rollback	0.90 s	88 mA	261 mJ	0.90 s	78 mA	232 mJ	0.89×

Note 1 — the window is the cost, not the packet.

One BLE advertisement is 0.99 mJ: *eighteen times cheaper* than an MQTT heartbeat. But a *delivered* heartbeat means holding an advertising window open long enough for the gateway to hear it — 1.5 s, at 208 mJ. **The radio is 7 % of that; the other 93 % is simply having the CPU switched on.** Optimise the window, not the packet.

Note 2 — BLE deletes the SDK’s worst battery failure mode.

On WiFi, a device that cannot reach its broker retries *forever* on a ladder that saturates at one attempt per minute (`nff_core.c:31`, hardcoded): 677 mJ each, or **500 mA h per month** — 8 % of a 6 000 mA h battery every month, with no telemetry to show for it. An advertise-only BLE node **has no link to fail**: an unheard advertisement costs exactly what a heard one costs, and nothing more. **This is arguably the strongest reliability argument for BLE here, and it is not a power-efficiency argument at all.**

Note 3 — BLE OTA costs more, and it does not matter.

2.40 mA h is 0.04 % of a 6 000 mA h battery. Twelve updates a year is 30 mA h: 0.6 % of the annual budget. **Ship the updates.** The 29 % penalty is real, worth knowing, and irrelevant to the decision.

4 What this means for a 6 000 mA h / 12-month node

Reference duty cycle: wake every 5 min, read a sensor, advertise, sleep. Budget: 5 100 mA h usable (85 % derate) over 8 760 h ⇒ **582 µA mean**, or **48.5 µA h per cycle**.

	CPU-active	Sleep floor	I_{mean}	Life
D0 naive, devkit	31.2 s	6.5 mA	10.8 mA	20 days — fails on the <i>sleep floor alone</i>
D1 naive, custom board	31.2 s	20 µA	5.01 mA	6.1 weeks — the literal spec, built the obvious way
D2 short wake	2.5 s	20 µA	380 µA	18.4 months — recommended; no extra parts
D3 30 s window, CPU light-slept	1.2 s	20 µA	394 µA	17.7 months — +32.768 kHz crystal

A 30 s CPU-active window cannot be made to work, on any radio. The duty cycle is 10 %, so the mean budget of 685 µA (un-derated) permits at most 6.85 mA while awake. **No ESP32 executes code at 6.85 mA** — Espressif’s floor is 20 mA to 31 mA (modem sleep, 80 MHz, radio *off*). **Over budget by ≥ 3× with a perfect battery and the radio switched off.**

But “awake for 30 s” and “the CPU runs for 30 s” are different requirements, and they differ by 12× in battery life. D3 keeps the full 30 s *advertising window* and runs the CPU for only 1.2 s of it, the rest in automatic light sleep with the BT controller holding the advertising timing. **Ceiling: the CPU may run for ≤ 3.9 s per 5 min cycle.**

The sleep floor is a board property, not a chip property. A devkit draws **6.5 mA asleep** (AMS1117 5 mA + power LED 1.4 mA + CP2102) — **11× the entire per-cycle budget, while doing nothing at all**. A custom board with a low- I_q LDO reaches **20 μ A**. No firmware recovers a devkit, and any battery-life figure measured on one is meaningless.

5 What the nff SDK would need

The BLE column above describes a device the nff SDK cannot currently build. Verified by reading the source, not inferred: (1) **there is no BLE** — `nff_port.h` is the SDK’s only hardware contract and exposes just `nff_port_mqtt_*` (MQTT over mTLS) and `nff_port_https_get_stream`; a repo-wide search for BLE, NimBLE, bluedroid, `esp_ble_*` and `advertis*` returns **zero hits in any firmware source**. (2) **There is no sleep** — no `esp_deep_sleep`, `esp_light_sleep` or `esp_pm_*` anywhere. The SDK’s entire low-power story is one sentence in `PROTOCOL.md`: `raise heartbeat_interval_s`.

The PAL is the clean seam, so this is an addition, not a rewrite: add `nff_port_ble_adv_*` and `nff_port_sleep_deep/light()`, an `nff_run_once()` entry point beside the never-exiting `nff_start_task()`, RTC.NOINIT state carry (the mechanism already exists in `nff_crash.c`), a packed 27-byte codec, and a gateway-side MQTT bridge.

Two hardcoded behaviours must also become configurable. The **reconnect ladder** (`nff_core.c:31`) saturates at 60s and retries forever — see note 2 above. And **SNTP** (`nff_port_esp32_arduino.c:60`) is unconditional and blocks `nff_connect()` for up to 20s on every cold boot: on a duty-cycled node that is **5× the entire CPU-active budget, every single wake**.

Payload constraint. BLE 4.2 legacy advertising carries **31 bytes** (**this part has no extended advertising — 31 B, not 255**). After manufacturer-data framing that is **27 bytes**. The nff heartbeat is a 250-byte JSON: it **does not fit, by 8×**, and the gateway must reconstruct it from a packed record (`ver1 + seq2 + flags1 + fw_id2 + batt1 + sensor16 + CMAC tag4 = 27`, exactly full).

6 Measured data — what is now real

Taken 2026-07-14 on the shunt rig (`datasheet.tex`), ESP32-WROOM-32E devkit, workload `esp32-nffload`. The rig measures the *whole devkit at its 5 V input*; every figure below is **de-embedded to the module’s 3.3 V rail** by subtracting the board overhead (6.5 mA: AMS1117 I_q + power LED + CP2102) and rescaling $E = 3.3\text{ V} \cdot I \cdot t$. Values marked * elsewhere in this sheet come from here.

Phase	t	I (board)	I (module)	E @ 3.3 V	Rate
Idle — radio off, CPU 80 MHz	6.2 s	41.5 mA	35.0 mA	0.72 J	$= I_{\text{core}}$
CPU flat out — radio off, 240 MHz	6.2 s	81.6 mA	75.1 mA	1.54 J	—
WiFi associate + DHCP (cold)	3.1 s	71.2 mA	64.7 mA	0.66 J	—
WiFi connected, idle	6.0 s	56.2 mA	49.7 mA	0.98 J	the always-on cost
Download 1 MiB (HTTP, radio only)	2.5 s	161 mA	154.6 mA	1.28 J	419 kB/s
Flash write 1 MiB (radio off)	6.8 s	95.0 mA	88.5 mA	1.99 J	154 kB/s
OTA 1 MiB download + write, concurrent	8.6 s	157 mA	150.5 mA	4.27 J	122 kB/s

The OTA is flash-bound, not radio-bound. The radio sustains 419 kB/s; the flash accepts only 154 kB/s. That is the single most consequential thing the measurement revealed, and it has two edges. **For WiFi it is good news:** the radio has 2.7× headroom, so the TLS and SHA-256 that a real nff OTA adds — and which this run did *not* include — eat radio headroom, not wall-clock, and the duration should barely move. **For BLE it is bad news:** at 15 kB/s BLE is radio-bound with no headroom at all, so the k_t penalty applies in full. **This is why k_t moved from 1.93 to 8.1 once the WiFi row stopped being a mock.**

What the measurement confirmed. $I_{\text{core}} = 35.0\text{ mA}$ against a modelled 35 mA, and the OTA mean current 150.5 mA against a modelled 155 mA to 158 mA. **The two-term current model in §2 is sound.** What was wrong was never the currents — it was the *durations*.

What is still *not* measured, and must not be read as if it were: every BLE figure (the SDK has no BLE, so there is nothing to flash); the sleep floors (below this rig’s noise — §7); and every nff-SDK-specific operation — mTLS, MQTT, the heartbeat, ECDSA verify — because the workload exercised generic WiFi, HTTP and flash, not the SDK itself. **Rig accuracy: relative figures are good to a few percent; absolute figures carry 10 % to 30 % because the shunt was never calibrated (R_{eff} assumed 1 Ω nominal; the resistor measures 1.1 Ω).**

7 Converting this sheet to measured data

Everything above rests on one quantity: I_{awake} , the module's mean current with the CPU running and the radio active. It is tens of mA — comfortably *inside* the range of the existing shunt rig (`datasheet.tex`). One afternoon converts this sheet.

1. **Calibrate — still outstanding.** Phase 0 has *not* been run, so the rig has no R_{eff} and will report a confident, wrong number. Run WIRECHECK too — wired to *nothing*, the rig once reported a perfectly plausible 590 mA.
2. Flash a bare BLE advertiser — no nff, no WiFi, no sensor. Mount per `datasheet.tex` (**the DUT's USB must not be connected** — it shorts the shunt). `nff power measure --for 30` $\Rightarrow I_{\text{awake}}$, and with it the whole BLE column.
3. Repeat with a WiFi/MQTT build $\Rightarrow$ the WiFi column, and the *real* k_I , which replaces the model in §2.
4. **The sleep floor cannot be measured on this rig.** At 10 μA across a 1 Ω shunt it is 10 μV , about 100 $\times$ below the noise floor — the rig's own datasheet marks deep sleep as out of scope. Use a Nordic PPK2 or an INA228. **Do not read the rig's silence as a zero:** a 6.5 mA devkit floor and a 20 μA custom-board floor are the difference between 20 days and 18 months, and the shunt rig returns indistinguishable noise for both.

Data status. Preliminary — typical values by engineering model; WiFi column modelled, BLE column derived from it per §2. Not production-tested. The 85 % battery derate additionally assumes a chemistry that can source the 130 mA radio pulse: Li-SOCl₂ cells passivate, and the worst pulse is the first one after a long sleep. **The ranking of operations and the feasibility verdict are robust to the modelling; the absolute figures are not.** Full derivation and error budget: `ble-power-budget.tex`. Rig specification: `datasheet.tex`.